

NyayKosh Backend — Codebase Analysis & Developer Guide

1. PROJECT OVERVIEW

• What is this project?

NyayKosh (meaning "Legal Repository / Dictionary") is a Django REST Framework (DRF) backend service that provides a Retrieval-Augmented Generation (RAG) legal assistant system.

• What problem does it solve?

Legal Indian statutes (such as the Constitution, Bharatiya Nyay Sanhita, Indian Contract Act, IT Act, etc.) are long, complex, and difficult for citizens or legal professionals to search through manually. NyayKosh indexes raw legal PDF documents into a local vector database (FAISS) and uses Google's Gemini LLM to answer legal questions grounded directly in official statutory context.

• What is the main purpose of the application?

1. Store user profile data and persistent chat history.
2. Index legal PDF documents into embeddings and FAISS index chunks.
3. Perform semantic similarity search across Indian legal documents.
4. Pass retrieved legal context to Google Gemini to generate authoritative, context-restricted answers.
5. Store conversation histories persistently in a JSONField database column.

• Who/what is the expected user or consumer?

Front-end clients (web or mobile apps, e.g., React/React Native apps) or external clients seeking AI-driven Indian legal assistance.

• What are the major components/modules?

1. backend/: Django core settings and URL routing.
2. User/: User management and chat session storage API.
3. rag/: Document chunking, FAISS vector indexing, semantic search, and Gemini LLM RAG pipeline.

2. PROJECT STRUCTURE

```
NyayKosh/
├── backend/
│   ├── backend/
│   │   ├── settings.py          # Core Django configuration & dependencies
│   │   ├── urls.py             # Main URL dispatcher & endpoint routing
│   │   ├── wsgi.py / asgi.py   # Web server entry points
│   │   └── User/
│   │       ├── models.py       # User and Chats database schemas
│   │       ├── views.py        # CRUD handlers for User and Chat models
│   │       ├── serializers.py  # DRF ModelSerializers for JSON conversion
│   │       ├── admin.py        # Django Admin interface registration
│   │       └── migrations/     # Database migration files
│   └── rag/
│       ├── views.py            # Core RAG engine: chunking, embedding, FAISS & LLM integration
│       ├── data/               # Raw legal PDF documents (BNS, Constitution, IT Act, etc.)
│       │   └── vector_db/      # (Generated) FAISS index (legal.index) & chunks pickle (chunks.pkl)
│       ├── db.sqlite3          # SQLite database file
│       ├── manage.py           # Django command-line runner
│       └── requirements.txt     # Python package requirements
```

Component Breakdown

- ``backend/backend/``: Base configuration module. Connects installed apps (``User``, ``rag``, ``rest_framework``, ``corsheaders``) and sets up root HTTP routing.
- ``backend/User/``: Manages user accounts and chat history storage.
- `models.py`: Defines `User` (profile details) and `Chats` (JSON structure storing conversational threads).
- `views.py`: REST API endpoints for user account CRUD and updating chat transcript lists.
- ``backend/rag/``: The RAG engine.
- `data/`: Holds 10 official Indian legal PDFs.
- `views.py`: Extracts text from PDFs using `PyMuPDF (fitz)`, creates 6000-character section-based chunks, computes vector embeddings via `sentence-transformers`, searches with `faiss`, and generates answers with `google-genai`.

3. TECHNOLOGY STACK

Library/Technology	Purpose	Where Used
Python 3	Core programming language	Entire backend codebase
Django 5.2.17	Core Web Framework	<code>`backend/backend/`</code> , <code>`User/`</code> , <code>`rag/`</code>
Django REST Framework (3.18.0)	API building blocks, <code>`@api_view`</code> , <code>`Response`</code> , <code>`serializers`</code>	<code>`User/views.py`</code> , <code>`rag/views.py`</code> , <code>`User/serializers.py`</code>
django-cors-headers (4.9.0)	Cross-Origin Resource Sharing middleware for frontend access	<code>`backend/settings.py`</code>
SQLite3	Relational Database	<code>`backend/db.sqlite3`</code>
PyMuPDF (<code>`fitz`</code>)	Extracting text from PDF legal documents	<code>`rag/views.py`</code> (<code>`create_embeddings`</code>)
Sentence-Transformers	Generating 384-dim text embeddings (<code>`all-MiniLM-L6-v2`</code>)	<code>`rag/views.py`</code> (<code>`model.encode()`</code>)
FAISS (<code>`faiss-cpu`</code>)	Fast similarity search on dense vector space (<code>`IndexFlatIP`</code>)	<code>`rag/views.py`</code> (<code>`create_embeddings`</code> , <code>`load_index`</code> , <code>`ask_question`</code>)
Google GenAI SDK (<code>`google-genai`</code>)	Accessing Gemini model (<code>`gemini-3.6-flash`</code>) for response generation	<code>`rag/views.py`</code> (<code>`ask_question`</code>)
Pickle	Serializing legal text chunks into <code>`*.pkl`</code> binary files	<code>`rag/views.py`</code> (<code>`create_embeddings`</code> , <code>`load_index`</code>)
Cloudinary	Included in requirements (intended for media storage, unintegrated)	<code>`requirements.txt`</code>

4. APPLICATION ENTRY POINT

- **HTTP Entry Point**: ``backend/manage.py`` starts the server when invoking ``python manage.py runserver``.
- **WSGI/ASGI Entry**: ``backend/backend/wsgi.py`` / ``asgi.py`` exposes the application interface for web servers like Gunicorn or Uvicorn.

- **Request Dispatcher:** ``backend/backend/urls.py`` maps HTTP requests to view functions in ``User/views.py`` and ``rag/views.py``.

- **Startup Sequence:**

1. `settings.py` loads configuration, middleware (CorsMiddleware, AuthenticationMiddleware), and `INSTALLED_APPS`.

2. `rag/views.py` executes top-level module code on import: instantiates `model = SentenceTransformer("all-MiniLM-L6-v2")` into memory.

5. DATA FLOW

User Creation & Chat Thread Initialization Flow

Client Request (POST /users/) → UserSerializer validation → User saved to SQLite → JSON Response

Client Request (POST /users/<user_id>/chats/) → ChatsSerializer validation → Chats saved with empty list [] → JSON Response

RAG Inquiry Data Flow (``ask_question``)

```

Input (JSON Payload)
  ■ { "question": "What is Section 302 BNS?", "chat_id": 1, "user_id": 1, "conversation_id": 0 }
  ▼
Validation (rag/views.py: ask_question)
  ■ Checks required fields.
  ■ Fetches Chats record: Chats.objects.get(id=chat_id, user_id=user_id).
  ■ Verifies 0 <= conversation_id < len(chat.chats).
  ▼
Vector Search (rag/views.py: load_index + sentence_transformers + FAISS)
  ■ Loads FAISS index (legal.index) & text chunks (chunks.pkl).
  ■ Encodes query string into normalized embedding vector using `all-MiniLM-L6-v2`.
  ■ Executes IndexFlatIP search for top k=5 nearest vectors.
  ▼
Context Construction (rag/views.py)
  ■ Concatenates matching text chunks into a context string.
  ■ Collects unique source document filenames.
  ▼
Gemini LLM Prompting (google-genai)
  ■ Formats prompt: "Answer the question using only the provided context...".
  ■ Calls model `gemini-3.6-flash`.
  ▼
State Persistence & Response
  ■ Appends tuple [query, response.text] to `chat.chats[conversation_id]`.
  ■ Saves updated `Chats` model to SQLite.
  ■ Returns HTTP 200: { "question": ..., "answer": ..., "sources": [...] }

```

6. CORE BUSINESS LOGIC

1. Structured Document Chunking (``rag/views.py: get_chunks``)

- **Importance:** High.

- Uses regex pattern `r'(?m)^\s*(\d+[A-Z]?)\.\s+(.+)?(?:\n|$)'` to detect section headers in legal texts (e.g., 12. Section Title).

- If regex matching fails, falls back to paragraph splitting (`\n\s*\n`).

- Merges subsections up to a length limit of 6,000 characters.

2. Vector Index Generation (`rag/views.py: create_embeddings``)

- Reads 10 PDF documents from `rag/data/`.
- Extracts plain text via PyMuPDF.
- Computes normalized embeddings with `SentenceTransformer("all-MiniLM-L6-v2")`.
- Creates an Inner Product FAISS index (`faiss.IndexFlatIP(384)`) for cosine similarity.
- Persists vector database to `rag/data/vector_db/legal.index` and metadata chunks to `chunks.pkl`.

3. Grounding & RAG Query Execution (`rag/views.py: ask_question``)

- Performs inner-product top-5 similarity search against FAISS index.
- Restricts LLM to output answers derived **only** from retrieved legal text.

4. JSON Chat History Persistence (`User/views.py: chatApi`` & `rag/views.py: ask_question``)

- Stores conversations in a nested array format: `chat.chats[conversation_id].append([query, answer])`.

7. FEATURES

Core Features

- **Legal Document RAG Engine:** Indexes PDFs and generates AI responses grounded in Indian legal text.
- **Persistent Conversational Chat Storage:** Stores list of question-answer pairs per chat thread in SQLite using Django `JSONField``.
- **User Profile Management:** REST API endpoints for user registration, updates, and deletion.

Supporting Features

- **Django Admin Interface:** Admin configuration registered for `User`` and `Chats`` models in `User/admin.py``.

8. API / ENDPOINTS

1. User Management (`/users/`` and `/users/<pk>/``)

- **File:** `User/views.py`` (`userApi``)
- **Methods:**
 - GET `/users/`: Lists all users.
 - GET `/users/<pk>/`: Gets user by ID.
 - POST `/users/`: Creates a user (Body: `{ "name", "age", "profession", "email", "password" }`).
 - PUT `/users/<pk>/`: Replaces user details.
 - PATCH `/users/<pk>/`: Partially updates user details.
 - DELETE `/users/<pk>/`: Deletes user.

2. Chat History Management (`/users/<user_id>/chats/`` and `/users/<user_id>/chats/<pk>/``)

- **File:** ``User/views.py`` (``chatApi``)

- **Methods:**

- GET `/users/<user_id>/chats/`: Lists chats belonging to user.

- GET `/users/<user_id>/chats/<pk>/`: Gets specific chat record.

- POST `/users/<user_id>/chats/`: Creates a new chat thread object (Body: `{ "chats": [ [] ] }`).

- PATCH `/users/<user_id>/chats/<pk>/`: Appends manual `[query, answer]` pair to `chat.chats[conversation_id]`.

- DELETE `/users/<user_id>/chats/<pk>/`: Deletes chat record.

3. RAG Question Answering (``ask_question`` view in ``rag/views.py``)

- **Method:** ``POST``

- **Payload:** `{ "question": string, "chat_id": int, "user_id": int, "conversation_id": int }`

- **Output:** `{ "question": string, "answer": string, "sources": [string] }`

- ***Note*:** View is defined with ``@api_view(["POST"])`` in ``rag/views.py``, but missing from ``backend/urls.py``.

9. DATABASE / DATA STORAGE

- **Database System:** SQLite (``db.sqlite3``).

- **Models:**

- User (`User/models.py`):

- `id`: Auto primary key (`BigAutoField`).

- `name`: `CharField(max_length=100)`.

- `age`: `CharField()`.

- `profession`: `CharField(max_length=50)`.

- `email`: `EmailField(max_length=254)`.

- `password`: `CharField(max_length=100)` (plain text string).

- Chats (`User/models.py`):

- `id`: Auto primary key (`BigAutoField`).

- `user`: `ForeignKey(User, on_delete=models.CASCADE)`.

- `chats`: `JSONField(default=list)`. Stores array of conversations, where each conversation is an array of `[query, answer]` pairs.

10. EXTERNAL SERVICES

1. Google Gemini API (``google-genai`` package)

- **Model:** `gemini-3.6-flash`

- **Where Called:** `rag/views.py` in `ask_question()`

- **Purpose:** Generates natural language answers constrained by retrieved legal PDF context.

- **Failure Handling:** Basic execution try/catch not wrapped around SDK call; uncaught exception will return DRF 500 error if API key fails or quota is exceeded.

11. AUTHENTICATION & AUTHORIZATION

- **Authentication:** None currently implemented.
- **Passwords:** Passwords are saved in plain text in ``User.password``.
- **Authorization:** Endpoint URLs accept ``user_id`` as a route parameter without token verification (e.g. JWT or Session).

12. ERROR HANDLING

- Custom HTTP 400 validation responses for missing parameters (``question``, ``chat_id``, ``user_id``, ``conversation_id``).
- HTTP 404 response if ``Chats.objects.get()`` raises ``Chats.DoesNotExist``.
- HTTP 400 response if vector index or chunks are missing on disk.
- Unhandled ``User.DoesNotExist`` exceptions in ``User/views.py`` will trigger Django 500 responses when invalid IDs are passed.

13. CONFIGURATION & ENVIRONMENT VARIABLES

- ``SECRET_KEY``: Hardcoded in ``backend/settings.py`` (Security risk in production).
- ``DEBUG``: Set to ``True`` in ``backend/settings.py``.
- ``DATA_PATH``: Inferred dynamically via ``settings.BASE_DIR / "rag" / "data"``.
- ``Gemini API Key``: Hardcoded string placeholder ``"enter api key"`` in ``rag/views.py``. Needs ``os.environ.get("GEMINI_API_KEY")``.

14. BACKGROUND JOBS / ASYNC PROCESSING

- Document embedding (``create_embeddings``) is currently synchronous.
- Embedding model loading (``SentenceTransformer``) happens synchronously on server startup when loading ``rag/views.py``.

15. TESTING

- ``User/tests.py`` and ``rag/tests.py`` contain empty stub test cases (``# Create your tests here.``).
- No unit tests or integration tests currently exist.

16. DEPENDENCIES AND ARCHITECTURE

- **Architecture:** Classic Django 3-Tier Layered Architecture (View - Model - Database) augmented with a local RAG vector search pipeline.
- **Pattern:** Function-Based Views (FBV) with DRF serializers.

17. IMPORTANT DESIGN DECISIONS

1. Local Sentence-Transformers & FAISS

- ***Choice*:** Run embedding locally on CPU (all-MiniLM-L6-v2 + faiss-cpu) rather than using external vector databases like Pinecone or Qdrant.
- ***Advantage*:** Zero cost, completely local vector search, no external vector cloud latency.
- ***Disadvantage*:** High memory usage on backend instance during startup and indexing.

2. JSONField for Chat History

- ***Choice*:** Store all messages as nested arrays inside a single JSONField column on the Chats model.
- ***Advantage*:** Quick retrieval of full chat history in one query.
- ***Disadvantage*:** Concurrent update race conditions when appending messages; requires loading and resaving entire JSON blob.

18. COMPLETE REQUEST / EXECUTION FLOW

```
User (HTTP POST /rag/ask/) [Proposed route]
  ▼
backend/urls.py -> rag/views.py (ask_question)
  ▼
Validation: Check request.data for 'question', 'chat_id', 'user_id', 'conversation_id'
  ▼
Database Lookup: Chats.objects.get(id=chat_id, user_id=user_id)
  ▼
RAG Vector Retrieval:
  ■■■ Load index & chunks (rag/data/vector_db/)
  ■■■ Embed query with SentenceTransformer("all-MiniLM-L6-v2")
  ■■■ faiss.IndexFlatIP.search() -> Top 5 legal text chunks
  ▼
LLM Generation:
  ■■■ google.genai.Client().models.generate_content(model="gemini-3.6-flash", prompt=prompt)
  ▼
Database State Update:
  ■■■ chat.chats[conversation_id].append([query, response.text])
  ■■■ chat.save()
  ▼
HTTP 200 Response: { "question": ..., "answer": ..., "sources": [...] }
```

19. CODEBASE MAP

Component	Responsibility	Inputs	Outputs
`backend/backend/urls.py`	Top-level HTTP routing	HTTP Requests	Dispatched View Functions
`User/views.py`	User CRUD & manual chat persistence	JSON payloads, route IDs	DRF `Response` objects
`User/models.py`	Defines DB Schema for User & Chats	ORM Operations	SQLite DB Rows
`rag/views.py`	PDF chunking, embedding, vector search, LLM RAG	PDF files, search questions	Answers with cited PDF sources

20. CRITICAL FILES TO READ FIRST

1. `backend/backend/urls.py` — Maps all API endpoints.
2. `User/models.py` — Database schema for users and chat logs.
3. `User/views.py` — CRUD handlers for user accounts and chat records.
4. `rag/views.py` — Core RAG logic, chunking regex, vector search, and Gemini LLM call.
5. `backend/backend/settings.py` — Project configuration and installed apps.

21. POTENTIAL ISSUES & RECOMMENDATIONS

1. Unrouted Endpoint (`rag.views.ask\_question`)

- ***Issue*:** `ask_question` is defined in `rag/views.py` but is **not** included in `backend/backend/urls.py`. Clients cannot call the RAG endpoint until `path('ask/', rag_views.ask_question)` is added to `urls.py`.

2. Hardcoded API Key

- ***Issue*:** `rag/views.py:148` sets `client = genai.Client(api_key="enter api key")`.

- ***Fix*:** Replace with `os.getenv("GEMINI_API_KEY")`.

3. Plain-Text Passwords

- ***Issue*:** `User/views.py` saves raw user passwords directly into `User.password`.

- ***Fix*:** Use Django's built-in `AbstractUser` model and `make_password()` / `check_password()`.

4. Invalid Field Argument in `User/models.py`

- ***Issue*:** Line 5 defines `age = models.CharField()`. In Django, `CharField` requires `max_length`.

5. No Exception Handling on DB Lookups

- ***Issue*:** `User/views.py` uses `User.objects.get(id=pk)` without `try/except` for `User.DoesNotExist`, raising uncaught 500 errors on invalid IDs.

22. FINAL SUMMARY

A. What this project does

NyayKosh is an Indian legal assistant AI backend built with Django REST Framework. It parses official Indian legal PDF documents, breaks them into structured section chunks, embeds them using Sentence Transformers, indexes them using FAISS, and uses Google Gemini (gemini-3.6-flash) to generate accurate, context-bound legal answers to user queries while keeping persistent conversation logs.

B. Architecture

A 3-tier Django REST Framework application integrated with an in-memory RAG pipeline combining PyMuPDF, Sentence Transformers (all-MiniLM-L6-v2), local FAISS vector index, and Google's Gemini SDK.

C. Main Data Flow

Client Question → DRF View → FAISS Semantic Search → Top 5 Context Chunks → Gemini 3.6 Flash Generation → Append to User's Chats .chats JSON column in SQLite → JSON Response.

D. Core Business Logic

Legal section parsing using regex, dense vector search via normalized inner-product FAISS indexing, prompt grounding for hallucination-free legal responses, and nested JSON conversation array persistence.

E. Top 5 Most Important Files

1. ``rag/views.py``
2. ``User/models.py``
3. ``User/views.py``
4. ``backend/urls.py``
5. ``backend/settings.py``

F. Top 5 Key Dependencies

1. ``Django` / `django-rest-framework``
2. ``google-generativeai``
3. ``sentence-transformers``
4. ``faiss-cpu``
5. ``PyMuPDF` ( `fitz` )`

How the Whole System Works in 2-3 Minutes (For Developers)

"NyayKosh is a Django backend that acts as an AI legal assistant for Indian law.

PDFs of Indian laws (like BNS, Constitution, IT Act) sit in ``rag/data/``. ``rag/views.py`` extracts their text, splits them into legal sections using regex, and uses ``sentence-transformers`` to turn them into 384-dimensional vector embeddings stored in a local ``faiss-cpu`` index (``legal.index``).

>

*When a user submits a legal question, ``ask_question`` computes the vector for the user's query, finds the top 5 most relevant legal text sections from FAISS, constructs a prompt forcing Gemini (``gemini-3.6-flash``) to answer using **only** that context, appends the QA pair into the user's ``Chats`` JSON thread in SQLite, and returns the answer along with cited PDF sources."*